Universidade da Beira Interior — Programação de Dispositivos Móveis

Protocolo Experimental

Persistência local e funcionamento offline

Grupo 7: João Loja, Ivan Bonifácio, Tomás Silva, Tomás Fonseca

Repositório: `https://github.com/jvasco19/Projeto-PDM`

---

1 Enquadramento teórico

1.1 Persistência local e funcionamento offline

Uma aplicação móvel não pode depender de uma ligação permanente à Internet, pois o utilizador pode estar num local sem cobertura, em modo avião ou com uma rede lenta. A persistência local consiste em guardar os dados no próprio dispositivo, de forma a que sobrevivam ao fecho da aplicação e estejam disponíveis rapidamente, sem depender da rede. O Android oferece várias opções de persistência, adequadas a tipos de dados diferentes. Existem pares chave-valor para definições simples, ficheiros para dados não estruturados e bases de dados relacionais para dados estruturados onde seja necessário pesquisar e filtrar.

1.2 SharedPreferences e DataStore

O **SharedPreferences** é a forma tradicional de guardar pequenos pares chave-valor no Android, armazenados num ficheiro XML na pasta privada da aplicação [2]. A primeira leitura é síncrona e pode bloquear a interface, e a interface não é atualizada automaticamente quando um valor muda, sendo necessário registar um *listener* para detetar alterações. O **DataStore** é o substituto recomendado. As escritas são assíncronas (funções `suspend`) e nunca ficam guardadas a meio. Ao ler um valor o DataStore devolve um `Flow`, que emite o novo valor sempre que este é alterado, mantendo a interface atualizada [3]. O DataStore foi concebido para Kotlin e, em Java, a alternativa é o SharedPreferences.

1.3 Ficheiros internos

Cada aplicação possui uma pasta privada (`filesDir`), inacessível às outras aplicações e que não requer permissões [4]. É adequada a dados não estruturados, como ficheiros JSON ou imagens, e só é apagada quando a aplicação é desinstalada. Já a pasta `cacheDir` pode ser limpa pelo sistema quando o dispositivo tem pouco espaço, pelo que não deve conter dados importantes.

1.4 SQLite e Room

O **SQLite** é a base de dados relacional incluída no Android [5]. A sua utilização direta, através da classe `SQLiteOpenHelper`, obriga a escrever o SQL em texto e a converter manualmente cada linha do resultado através de um `Cursor`. Além disso, os erros de SQL só são detetados em tempo de execução.

O **Room** é a biblioteca recomendada para trabalhar com o SQLite [6]. Não é uma base de dados diferente. Funciona como uma camada de abstração sobre ele, composta por três elementos:

- **Entity**: classe anotada com `@Entity`, que define uma tabela, em que cada propriedade corresponde a uma coluna;
- **DAO** (*Data Access Object*): interface anotada com `@Dao`, onde se declaram as operações sobre os dados (`@Insert`, `@Query`, `@Update`, `@Delete`);
- **Database**: classe abstrata anotada com `@Database`, que indica as entidades, a versão do *schema* e disponibiliza os DAOs. Deve existir uma única instância em toda a aplicação.

A partir destas anotações, o Room gera durante a compilação o código que cria as tabelas, executa as consultas e converte os resultados em objetos. As consultas SQL são verificadas antes da aplicação ser executada, assim algum erro no SQL é apanhado em tempo de compilação. As funções de escrita são declaradas com `suspend` e executadas em segundo plano, visto que o Room não permite aceder à base de dados na *main thread*. As consultas que devolvem um `Flow` emitem novos resultados sempre que a tabela é alterada [9].

1.5 Migrações da Base de Dados

Quando uma nova versão da aplicação altera o *schema*, a base de dados já instalada no dispositivo mantém a estrutura antiga. O Room deteta a diferença através da comparação de uma *hash* do *schema* guardada na própria base de dados com a *hash* do *schema* da nova versão. Para atualizar a estrutura da base de dados, é necessário fazer uma `migração`, ou seja, utilizar uma classe `Migration` que define o SQL necessário para passar de uma versão à seguinte, preservando os dados já existentes [7]. Em alterações simples, o Room pode gerar a migração automaticamente (`AutoMigration`), desde que o *schema* de cada versão seja exportado em JSON. Existe ainda a migração destrutiva, que apaga e recria a base de dados, perdendo todos os dados, pelo que só deve ser usada em desenvolvimento e não em produção.

1.6 Repository e estratégia offline-first

O padrão **Repository** centraliza o acesso aos dados numa única classe, pelo que a interface não sabe se os dados vêm de uma base de dados, de um ficheiro ou de um servidor [1]. Na estratégia **offline-first**, a aplicação trabalha sempre com a base de dados local. A interface do utilizador lê sempre os dados locais, as escritas são feitas primeiro localmente e a sincronização com um servidor, quando existe, decorre em segundo plano [8]. Quando há sincronização, alterações feitas sem ligação em vários dispositivos podem originar conflitos, que é necessário resolver com uma estratégia definida como, por exemplo, *last-write-wins*.

2 Objetivos

Este protocolo tem como objetivo verificar, na aplicação *Gestor de Tarefas* desenvolvida pelo grupo, os pontos seguintes:

O1. As operações CRUD sobre tarefas são guardadas numa base de dados Room e persistem após o fecho da aplicação.

O2. Uma definição guardada com DataStore (“Mostrar concluídas”) persiste entre sessões e altera os dados apresentados.

O3. A aplicação grava corretamente um ficheiro JSON (`backup.json`) no armazenamento interno privado.

O4. Todas as funcionalidades se mantêm operacionais sem ligação à Internet.

3 Configuração utilizada

3.1 Ambiente

Item	Valor
IDE	Android Studio Quail 4 (2026.1.4)
Android Gradle Plugin	9.4.1
Linguagem / interface	Kotlin + layouts XML (ViewBinding)
Bibliotecas	Room 2.8.5 (com KSP 2.3.6), DataStore Preferences 1.1.7
`minSdk` / `targetSdk`	29 / 37
Dispositivo de teste	Emulador *Medium Phone*, Android 17.0 (API 37.1)
Package da aplicação	`com.grupo7.gestordetarefas`
Código-fonte	`https://github.com/jvasco19/Projeto-PDM` (ramo `main`) [11]

Tabela 1. Configuração do ambiente de teste.

3.2 Componentes da aplicação

Componente	Implementação
Entity	`data/Task.kt`: tabela `tasks`
DAO	`data/TaskDao.kt`: `observeAll()`, `getAll()`, `insert`, `update`, `delete`
Database	`data/AppDatabase.kt`: regista a entidade `Task` e disponibiliza o `TaskDao`, com instância única criada em `TarefasApp.kt`
DataStore	`data/SettingsStore.kt`: chave `show_done`
Ficheiros internos	`data/BackupStore.kt`: `filesDir/backup.json`
Repository	`data/TaskRepository.kt`: combina as tarefas (Room) com a definição (DataStore)
Application	`TarefasApp.kt`: cria uma única instância da base de dados, do DataStore, do backup e do Repository
Interface	`MainActivity.kt` com o layout em `activity_main.xml`

Tabela 2. Correspondência entre conceitos e ficheiros do projeto.

A aplicação implementa o Room (Entity, DAO e Database), o DataStore para a definição “Mostrar concluídas”, a escrita de um ficheiro JSON no armazenamento interno e o padrão Repository, seguindo os padrões recomendados na documentação oficial do Android [3, 4, 6]. O SharedPreferences, o SQLite utilizado diretamente e as migrações não estão implementados e são abordados apenas do ponto de vista teórico.

3.3 Estrutura da base de dados

A tabela é definida pela Entity seguinte e o schema gerado pelo Room encontra-se em `app/schemas/com.grupo7.gestordetarefas.data.AppDatabase/1.json`.

```
@Entity(tableName = "tasks")
data class Task(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val title: String,
    val done: Boolean = false,
    val createdAt: Long = System.currentTimeMillis()
)
```

Excerto 1. Código do ficheiro `Task.kt`.

Coluna	Tipo SQLite	Nulo	Observações
`id`	INTEGER	não	chave primária, `AUTOINCREMENT`
`title`	TEXT	não	texto da tarefa
`done`	INTEGER	não	0 = por fazer, 1 = concluída
`createdAt`	INTEGER	não	data de criação (milissegundos desde 1/1/1970)

Tabela 3. Colunas da tabela `tasks`.

3.4 Localização dos dados no dispositivo

- Base de dados: `/data/data/com.grupo7.gestordetarefas/databases/tarefas.db`
- DataStore: `/data/data/com.grupo7.gestordetarefas/files/datastore/settings.preferences_pb`
- Backup: `/data/data/com.grupo7.gestordetarefas/files/backup.json`

4 Dados de entrada

- **Estado inicial:** aplicação sem dados (ver passo 5 da secção 5.1).
- **Tarefas a criar, por esta ordem:** “Tarefa A”, “Tarefa B”, “Tarefa C”.
- **Tarefa a atualizar:** “Tarefa B” passa a “Tarefa B editada”.
- **Tarefa a concluir:** “Tarefa A”.
- **Tarefa a eliminar:** “Tarefa C”.
- **Tarefa a criar sem ligação à Internet:** “Tarefa D”.

5 Procedimento

5.1 Preparação

1. Clonar o repositório com `git clone https://github.com/jvasco19/Projeto-PDM`.
2. Abrir a pasta no Android Studio e aguardar a sincronização do Gradle.
3. Criar ou iniciar um emulador (*Device Manager*) com Android 10 (API 29) ou superior, necessário para instalar a aplicação.
4. Executar a aplicação (*Run*).
5. Garantir o estado inicial sem dados. No emulador, abrir *Definições* → *Apps* → *Gestor de Tarefas* → *Armazenamento e cache* → *Limpar armazenamento*, e voltar a abrir a aplicação.
6. Abrir *View* → *Tool Windows* → *App Inspection* → *Database Inspector*, selecionar o processo `com.grupo7.gestordetarefas` e ativar *Live updates* [10].

5.2 Operações CRUD (Room)

1. Adicionar as três tarefas dos dados de entrada, pela ordem indicada. Para cada uma, escrever o nome no campo de texto e carregar em **Adicionar**.
2. Tocar em "Tarefa B", alterar o texto para "Tarefa B editada" e carregar em **Atualizar**.
3. Tocar em "Tarefa A" e carregar em **Concluir**.
4. Tocar em "Tarefa C" e carregar em **Eliminar**.
5. Com o campo vazio, carregar em **Adicionar** e, sem nenhuma tarefa selecionada, carregar em **Atualizar**. Registar as mensagens apresentadas no ecrã (M2).
6. No Database Inspector, registar o número de linhas da tabela `tasks` e os valores de `title` e `done` de cada uma (M1).

5.3 Persistência após reinício

1. Fechar a aplicação a partir do ecrã de aplicações recentes (deslizar para fora) e voltar a abri-la.
2. Registar as tarefas apresentadas (M3).

5.4 Definição com DataStore

1. Desligar o interruptor **Mostrar concluídas** e registar as tarefas apresentadas (M4).
2. Fechar a aplicação a partir dos recentes e voltar a abri-la.
3. Registar o estado do interruptor e as tarefas apresentadas (M5).

5.5 Ficheiros internos

1. Com o interruptor ainda desligado, carregar em **Exportar backup** e registar a mensagem apresentada (M6).
2. Abrir *View* → *Tool Windows* → *Device Explorer*, navegar até `/data/data/com.grupo7.gestordetarefas/files/` (*Synchronize*, se necessário) e abrir o `backup.json`.
3. Comparar o conteúdo do ficheiro com a tabela `tasks` (M7).

5.6 Funcionamento sem ligação à Internet

1. Voltar a ligar o interruptor **Mostrar concluídas**.
2. Ativar o modo de avião no emulador (barra de notificações).
3. Adicionar a tarefa "Tarefa D", concluí-la e voltar a marcá-la como por fazer (M8).
4. Fechar a aplicação a partir dos recentes, voltar a abri-la e registar as tarefas apresentadas (M9).

5.7 Estabilidade

1. Durante todo o procedimento, manter aberto o *Logcat* com o filtro `package:mine`.
2. No fim, pesquisar `FATAL EXCEPTION` (*Ctrl+F/Cmd+F*) e verificar o número de ocorrências (M10).

6 Métricas

Id	Métrica	Como medir
M1	Número de linhas na tabela `tasks` após a secção 5.2	Database Inspector
M2	Mensagens de validação apresentadas no passo 5 da secção 5.2	Ecrã (mensagem)
M3	Tarefas apresentadas após reiniciar a aplicação	Ecrã
M4	Tarefas apresentadas com o interruptor desligado	Ecrã
M5	Estado do interruptor após reiniciar a aplicação	Ecrã
M6	Número de tarefas indicado na mensagem de backup	Ecrã (mensagem)
M7	Número de objetos JSON no ficheiro e correspondência com a tabela	Device Explorer
M8	Operações CRUD concluídas em modo de avião	Ecrã + Database Inspector
M9	Linhas na tabela após reiniciar em modo de avião	Ecrã + Database Inspector
M10	Número de erros fatais (`FATAL EXCEPTION`)	Logcat

Tabela 4. Métricas recolhidas.

7 Resultados esperados

Critério de sucesso. A aplicação cumpre os objetivos se todos os valores obtidos coincidirem com os esperados.

Id	Resultado esperado
M1	2 linhas na tabela `tasks`, `id` 1 “Tarefa A” (`done` = 1) e `id` 2 “Tarefa B editada” (`done` = 0). A “Tarefa C” (`id` 3) já não existe
M2	Mensagens “Escreve uma tarefa primeiro” e “Seleciona uma tarefa na lista”. A tabela continua com as mesmas 2 linhas
M3	Lista com “Tarefa B editada” e “Tarefa A”, por esta ordem, com “Tarefa A” assinalada com ✓
M4	Lista apenas com “Tarefa B editada”. A “Tarefa A” fica oculta, mas continua na tabela
M5	Interruptor **Mostrar concluídas** desligado e lista apenas com “Tarefa B editada”
M6	Mensagem “Backup de 2 tarefas guardado em backup.json”, que inclui a “Tarefa A” oculta
M7	`backup.json` com 2 objetos, `id` 1 “Tarefa A” (`done` = `true`) e `id` 2 “Tarefa B editada” (`done` = `false`), com os mesmos valores da tabela
M8	“Tarefa D” criada com `id` 4, marcada como concluída e de novo por fazer, sem mensagens de erro. No fim tem `done` = 0
M9	3 linhas na tabela e lista com “Tarefa D”, “Tarefa B editada” e “Tarefa A” (✓), por esta ordem
M10	0 ocorrências de `FATAL EXCEPTION`

Tabela 5. Resultados esperados por métrica.

8 Limitações

- Os dados existem apenas no dispositivo, porque a aplicação não implementa sincronização com um servidor, pelo que os dados não são partilhados entre dispositivos.
- A base de dados, o DataStore e o `backup.json` são apagados ao desinstalar a aplicação e o backup não pode ser importado de volta.

Referências

[1] Android Developers. *Guide to app architecture*. `https://developer.android.com/topic/architecture`.

[2] Android Developers. *Save simple data with SharedPreferences*. `https://developer.android.com/training/data-storage/shared-preferences`.

[3] Android Developers. *DataStore*. `https://developer.android.com/topic/libraries/architecture/datastore`.

[4] Android Developers. *Access app-specific files*. `https://developer.android.com/training/data-storage/app-specific`.

[5] Android Developers. *Save data using SQLite*. `https://developer.android.com/training/data-storage/sqlite`.

[6] Android Developers. *Save data in a local database using Room*. `https://developer.android.com/training/data-storage/room`.

[7] Android Developers. *Migrate your Room database*. `https://developer.android.com/training/data-storage/room/migrating-db-versions`.

[8] Android Developers. *Build an offline-first app*. `https://developer.android.com/topic/architecture/data-layer/offline-first`.

[9] Android Developers. *Kotlin coroutines on Android*. `https://developer.android.com/kotlin/coroutines`.

[10] Android Developers. *Debug your database with the Database Inspector*. `https://developer.android.com/studio/inspect/database`.

[11] Grupo 7. *Gestor de Tarefas – Persistência local e funcionamento offline* (repositório Git). `https://github.com/jvasco19/Projeto-PDM`. Ramo `main`.